

Министерство образования и науки РФ
федеральное государственное автономное образовательное учреждение высшего образования
«Омский государственный технический университет»

Факультет (институт) Информационных технологий и компьютерных систем
Кафедра Информатики и вычислительной техники

Отчётная работа

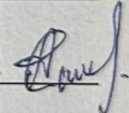
по дисциплине Учебная практика
на тему Разработка программ анализа данных

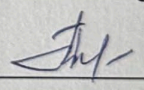
Пояснительная записка

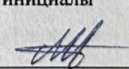
Студента (ки) Шмидта Антона Владиславовича
фамилия, имя, отчество полностью

Курс 1 Группа ИВТ-244

Направление (специальность) 09.03.01 –
Информатика и вычислительная техника
код, наименование

Руководитель От организации Симоненко А.Е. 
ученая степень, звание

От университета Вахний Т.В. 
фамилия, инициалы

Выполнил (а) 03.04.2025 
дата, подпись студента (ки)

К защите _____
дата, подпись руководителя

Выполнение и подготовка к защите отчёта по практике	Защита отчёта по практике	Итоговый рейтинг
55	40	95

Проект (работа) защищен (а) с оценкой отлично

Омск 2025

Содержание

Введение	3
1. РЕФЕРАТ ПО ТЕМЕ ТЕСТИРОВАНИЕ ПО.....	4
1.1 Цели и задачи тестирования программного обеспечения.....	4
1.2 Классификация и типология тестирования.....	5
1.3 Методологии и подходы к тестированию	5
1.4 Инструменты и автоматизация тестирования.....	6
1.5 Заключение	6
2 ПРОГРАММИРОВАНИЕ В ОБЪЕКТНО-ОРИЕНТИРОВАННОМ СТИЛЕ....	8
2.1 Описание хода выполнения работы.....	8
2.2 Вывод по разработке приложения	12
3 РЕШЕНИЕ ЗАДАЧ ПО АНАЛИЗУ ДАННЫХ.....	13
3.1 Описание хода выполнения работы.....	14
3.2 Вывод по разработке приложения	16
Заключение	17
Список использованных источников	18
Приложение А.....	19
1. Код объектно-ориентированного приложения	19
2. Код первой задачи для анализа данных.....	22
3. Код второй задачи для анализа данных	22

Введение

Отчёт содержит 18 с., 9 рис., 8 ист., 1 прил.

Практика – это одна из важнейших составляющих профессиональной подготовки специалистов. Цель практики по получению первичных профессиональных умений и навыков состоит в применении полученных знания и умения из области теории в область повседневной профессиональной деятельности. А также с целью приобретения первоначального практического опыта и подготовки будущего специалиста к самостоятельной трудовой деятельности.

Реализация данной цели предусматривает решение следующих задач:

- закрепление и углубление теоретических и практических знаний по программированию на Python;
- приобретение практических профессиональных умений и навыков по избранной специальности;
- развитие и углубление навыков программирования.

В процессе работы проводились исследования отдельных приложений для разработки программ, такие как PyCharm и Visual Studio Code.

Прохождение учебной практики было реализовано на базе ФГБОУ ВО «ОмГТУ» с 23 июня по 5 июля 2025 года.

1. РЕФЕРАТ ПО ТЕМЕ ТЕСТИРОВАНИЕ ПО

Тестирование программного обеспечения занимает одно из центральных мест в процессе разработки современных программных систем. По мере усложнения технологий и расширения применения программных продуктов в критически важных сферах — таких как медицина, транспорт, финансы — возрастает необходимость обеспечения их надёжности. Тестирование рассматривается не как заключительный этап разработки, а как неотъемлемая часть жизненного цикла программного обеспечения. Оно представляет собой систематизированный процесс выявления и устранения ошибок до их попадания к конечному пользователю.

Разработка программных продуктов без систематического тестирования приводит к появлению скрытых дефектов, которые могут проявиться в самый неожиданный момент. Именно поэтому уже на ранних этапах проектирования формулируются стратегии тестирования, определяются уровни покрытия и критерии приёмки. Такой подход позволяет строить надёжные и устойчивые к ошибкам системы, особенно если речь идёт о массовом или промышленном программном обеспечении, где сбои недопустимы.

1.1 Цели и задачи тестирования программного обеспечения

Основная цель тестирования — обеспечение качества программного продукта. Однако под этим термином скрывается целый комплекс задач, связанных с проверкой, верификацией и валидацией. Качество невозможно измерить напрямую, поэтому используются разнообразные метрики, такие как количество обнаруженных дефектов, плотность ошибок, стабильность версий, процент покрытия кода тестами.

Важно понимать, что тестирование не гарантирует отсутствие ошибок, но значительно снижает вероятность их возникновения при эксплуатации. За счёт системного подхода удаётся не только зафиксировать сбои, но и выявить закономерности, приводящие к ошибкам, что, в свою очередь, позволяет совершенствовать архитектуру продукта. Тестирование также служит инструментом оценки зрелости разработки, особенно в распределённых командах, где взаимодействие между разработчиками, аналитиками и тестировщиками критично.

1.2 Классификация и типология тестирования

Тестирование программного обеспечения классифицируется по множеству признаков, каждый из которых отвечает на свой вопрос: что именно проверяется, на каком этапе, в каком объёме, каким способом. Выделяются уровни тестирования: модульное (*unit testing*), интеграционное, системное и приёмочное. Каждый уровень решает определённые задачи и опирается на результаты предыдущего.

Модульное тестирование фокусируется на отдельных функциях и методах. Оно требует изоляции логики и независимости от внешних зависимостей. Для этого применяются моки, заглушки и шпионы. Интеграционное тестирование обеспечивает связность компонентов. Оно особенно важно при использовании микросервисной архитектуры, где взаимодействие между сервисами может вызывать сложности, несмотря на корректную работу каждой части в отдельности.

Системное тестирование рассматривает продукт как единое целое, проверяя его на соответствие функциональным и нефункциональным требованиям. Приёмочное тестирование позволяет подтвердить, что продукт готов к эксплуатации, и проводится либо пользователем, либо внешней стороной по согласованному регламенту.

Среди других классификаций выделяются функциональное и нефункциональное тестирование, позитивное и негативное, ручное и автоматическое, регрессионное, нагрузочное, стрессовое и многие другие виды. Их выбор зависит от специфики проекта и рисков, которые необходимо минимизировать.

1.3 Методологии и подходы к тестированию

Методологии тестирования представляют собой обобщённые принципы, по которым организуется работа в рамках проекта. Их выбор зависит от модели жизненного цикла ПО. В классических каскадных (*waterfall*) моделях тестирование следует за разработкой. В гибких (*agile*) методологиях, таких как *Scrum* или *Kanban*, тестирование проводится итеративно и тесно связано с непрерывной поставкой ценности пользователю.

В рамках *DevOps*-подхода тестирование становится частью конвейера разработки, тесно интегрируется в *CI/CD* и охватывает не только функциональные, но и эксплуатационные характеристики продукта. Здесь

критично автоматическое тестирование, которое обеспечивается на всех этапах — от проверки синтаксиса до нагрузочных тестов в *production*-среде.

Существуют также подходы, ориентированные на поведение (*BDD*) и тестирование через разработку (*TDD*), где процесс написания тестов опережает написание самого кода. Такие методы позволяют разработчикам сфокусироваться на требованиях и конечных результатах, обеспечивая высокий уровень надёжности с самого начала.

1.4 Инструменты и автоматизация тестирования

Автоматизация позволяет реализовать принципы воспроизводимости, скорости и надёжности тестов. В современных условиях трудно представить качественную разработку без тестов, выполняемых автоматически после каждого коммита. Это значительно ускоряет обратную связь и позволяет оперативно реагировать на изменения в коде.

Python предоставляет богатый инструментарий для автоматизированного тестирования. Помимо *unittest* и *pytest*, в арсенал входят *nose2*, *hypothesis* для *property – based* тестирования, *tox* для проверки совместимости с разными версиями интерпретатора. В крупных проектах применяются также средства мониторинга, отчётности (*Allure*, *TestRail*) и интеграции с системами контроля версий и *CI*-серверами (*Jenkins*, *GitHub Actions*, *GitLab CI*).

Отдельного внимания заслуживает тестирование пользовательских интерфейсов, где используются такие инструменты, как *Selenium*, *Playwright* или *PyAutoGUI*. Они позволяют имитировать действия пользователя и проводить *end – to – end* тестирование, включая сценарии с переходами, вводом данных и проверкой визуального результата.

1.5 Заключение

Таким образом, тестирование программного обеспечения представляет собой многоуровневый и многоаспектный процесс, от которого зависит качество, надёжность и коммерческий успех программного продукта. Его интеграция в жизненный цикл разработки позволяет систематически управлять качеством, уменьшать количество дефектов, ускорять вывод продукта на рынок и снижать затраты на сопровождение.

С развитием технологий и методов тестирования продолжает эволюционировать, внедряя элементы машинного обучения, интеллектуального

анализа логов и метрик, а также автоматического выявления аномалий. В современных условиях грамотное тестирование — это не только техническая задача, но и стратегическая инвестиция в стабильность и конкурентоспособность программного решения.

2 ПРОГРАММИРОВАНИЕ В ОБЪЕКТНО-ОРИЕНТИРОВАННОМ СТИЛЕ

Задание: Создайте класс АВТОМОБИЛЬ с методами, позволяющими вывести на экран информацию об автомобиле, а также определить, подходит ли данный автомобиль для заданных условий. Создайте дочерние классы: ЛЕГКОВОЙ_АВТОМОБИЛЬ (марка, модель, год выпуска, тип кузова), ГРУЗОВОЙ_АВТОМОБИЛЬ (марка, модель, год выпуска, грузоподъемность), АВТОБУС (марка, модель, год выпуска, количество мест) со своими методами вывода информации на экран и определения соответствия заданным условиям. Создайте список автомобилей, выведите полную информацию из базы на экран, а также организуйте поиск автомобилей с заданной маркой или годом выпуска.

2.1 Описание хода выполнения работы

```
class Car: 3 usages  3 ppljc
    def __init__(self, brand, model, year): 3 ppljc
        self.brand = brand
        self.model = model
        self.year = year

    def display_info(self): 3 usages  3 ppljc
        info = f"Brand: {self.brand}, Model: {self.model}, Year: {self.year}"
        return info

    def matches_conditions(self, **conditions): 3 usages  3 ppljc
        matches = True
        if 'brand' in conditions and conditions['brand'].lower() != self.brand.lower():
            matches = False
        if 'model' in conditions and conditions['model'].lower() != self.model.lower():
            matches = False
        if 'year' in conditions and conditions['year'] != self.year:
            matches = False
        return matches
```

Рисунок 1 – код класса АВТОМОБИЛЬ.

На рисунке 1 представлена реализация класса АВТОМОБИЛЬ, которая включает в себя задание внутренних параметров (переменных): марка, модель и год выпуска. Класс позволяет с помощью функции *display_info* выводить всю информацию об автомобиле. С использованием *matches_conditions* становится возможной проверка на совпадение определенного в классе автомобиля с заданным словарём, с выбранными полями.

В последующих классах, которые будут дочерними к классу на рисунке 1,

будет использоваться метод *super()*, позволяющий наследовать объекты родительского класса.

```
class PassengerCar(Car): 2 usages  3 ppljc
    def __init__(self, brand, model, year, body_type): 3 ppljc
        super().__init__(brand, model, year)
        self.body_type = body_type

    def display_info(self): 3 usages  3 ppljc
        base_info = super().display_info()
        return f"{base_info}, Body Type: {self.body_type}"

    def matches_conditions(self, **conditions): 3 ppljc
        matches = True
        if not super().matches_conditions(**conditions):
            matches = False
        if 'body_type' in conditions and conditions['body_type'].lower() != self.body_type.lower():
            matches = False
        return matches
```

Рисунок 2 – код класса ЛЕГКОВОЙ_АВТОМОБИЛЬ.

На рисунке 2 представлен унаследованный от класса АВТОМОБИЛЬ класс ЛЕГКОВОЙ_АВТОМОБИЛЬ. Ключевые изменения состоят в том, что помимо параметров родительского класса, добавился параметр типа кузова. Функция вывода информации была дополнена для вывода типа кузова. А сравнение с переданными условиями получило новое условие в виде типа кузова.

```
class Truck(Car): 2 usages  3 ppljc
    def __init__(self, brand, model, year, load_capacity): 3 ppljc
        super().__init__(brand, model, year)
        self.load_capacity = load_capacity # in tons

    def display_info(self): 3 usages  3 ppljc
        base_info = super().display_info()
        return f"{base_info}, Load Capacity: {self.load_capacity} tons"

    def matches_conditions(self, **conditions): 3 ppljc
        matches = True
        if not super().matches_conditions(**conditions):
            matches = False
        if 'min_load_capacity' in conditions and self.load_capacity < conditions['min_load_capacity']:
            matches = False
        return matches
```

Рисунок 3 – код класса ГРУЗОВОЙ_АВТОМОБИЛЬ.

На рисунке 3 показан унаследованный от класса АВТОМОБИЛЬ класс ГРУЗОВОЙ_АВТОМОБИЛЬ. Помимо параметров родительского класса, здесь появился параметр грузоподъемности. Функция вывода информации была дополнена для вывода грузоподъемности. А сравнение с переданными условиями получило новое условие – грузоподъемность.

```

class Bus(Car): 2 usages  🐍 ppljc
    def __init__(self, brand, model, year, seat_count):  🐍 ppljc
        super().__init__(brand, model, year)
        self.seat_count = seat_count

    def display_info(self): 3 usages  🐍 ppljc
        base_info = super().display_info()
        return f"{base_info}, Seat Count: {self.seat_count}"

    def matches_conditions(self, **conditions):  🐍 ppljc
        matches = True
        if not super().matches_conditions(**conditions):
            matches = False
        if 'min_seat_count' in conditions and self.seat_count < conditions['min_seat_count']:
            matches = False
        return matches

```

Рисунок 4 – код класса АВТОБУС.

На рисунке 4 представлен унаследованный от класса АВТОМОБИЛЬ класс АВТОБУС. Изменения: помимо параметров родительского класса добавился параметр вместимости (количество человек). Функция вывода информации была дополнена для вывода вместимости. А сравнение с переданными условиями получило новое условие в виде вместимости.

```

vehicles = [
    PassengerCar( brand: "Audi", model: "SQ7", year: 2018, body_type: "SUV"),
    PassengerCar( brand: "Volkswagen", model: "Passat", year: 2023, body_type: "sedan"),
    Truck( brand: "Volvo", model: "FH16", year: 2015, load_capacity: 20),
    Truck( brand: "Scania", model: "R450", year: 2019, load_capacity: 18),
    Bus( brand: "Mercedes", model: "Sprinter", year: 2017, seat_count: 20),
    Bus( brand: "Volvo", model: "B9R", year: 2014, seat_count: 50)
]

print("Все машины:")
for vehicle in vehicles:
    print(vehicle.display_info())

search_brand = "Volvo"
print(f"\nМашины бренда - '{search_brand}':")
for vehicle in vehicles:
    if vehicle.brand.lower() == search_brand.lower():
        print(vehicle.display_info())

search_year = 2019
print(f"\nМашины года - {search_year}:")
for vehicle in vehicles:
    if vehicle.year == search_year:
        print(vehicle.display_info())

```

Рисунок 5 – инициализация и проверка работы классов.

На рисунке 5 показано создание списка из 6 автомобилей. Затем вывод данного списка. Потом проверен поиск среди машин по марке и по году выпуска.

```

Все машины:
Brand: Audi, Model: SQ7, Year: 2018, Body Type: SUV
Brand: Volkswagen, Model: Passat, Year: 2023, Body Type: sedan
Brand: Volvo, Model: FH16, Year: 2015, Load Capacity: 20 tons
Brand: Scania, Model: R450, Year: 2019, Load Capacity: 18 tons
Brand: Mercedes, Model: Sprinter, Year: 2017, Seat Count: 20
Brand: Volvo, Model: B9R, Year: 2014, Seat Count: 50

Машины бренда - 'Volvo':
Brand: Volvo, Model: FH16, Year: 2015, Load Capacity: 20 tons
Brand: Volvo, Model: B9R, Year: 2014, Seat Count: 50

Машины года - 2019:
Brand: Scania, Model: R450, Year: 2019, Load Capacity: 18 tons

```

Рисунок 6 – результат работы программы.

На рисунке 6 показан вывод при запуске программы.

2.2 Вывод по разработке приложения

В ходе разработки было создано простое объектно-ориентированное приложение для представления информации об автомобилях различных типов – легковых, грузовых и автобусах.

Родительский класс АВТОМОБИЛЬ содержит общие параметры: *brand*, *model*, *year*, что позволяет избежать дублирования кода. Дочерние классы: ЛЕГКОВОЙ_АВТОМОБИЛЬ, ГРУЗОВОЙ_АВТОМОБИЛЬ и АВТОБУС – расширяют базовый функционал, добавляя специфические свойства: *body_type*, *load_capacity*, *seat_count* соответственно.

Метод *display_info()* обеспечивает вывод информации об объекте в удобной форме, а метод *matches_conditions()* реализует фильтрацию по заданным критериям, например: по году выпуска, бренду или другим параметрам.

Программа демонстрирует основные принципы объектно-ориентированного программирования:

- наследование – для переиспользования общей логики,
- переопределение методов – для вывода данных, характерных для каждого типа автомобиля,
- инкапсуляция – для хранения и обработки данных внутри объектов,
- полиморфизм – для вызова методов *display_info()* и *matches_conditions()* без знания точного типа объекта.

Такой подход делает код удобным для сопровождения и расширения. В будущем можно легко добавить новые типы транспортных средств или новые критерии фильтрации, не нарушая архитектуру программы.

Я научился разрабатывать модели компонентов информационных систем, включая модели баз данных, и модели интерфейсов «человек – электронно-вычислительная машина».

3 РЕШЕНИЕ ЗАДАЧ ПО АНАЛИЗУ ДАННЫХ

Задача: Считайте в переменные *tr_mcc_codes*, *tr_types*, *transactions* и *customers_gender_train* из одноимённых таблиц из папки *data*. В *transactions* считайте только первые 1000000 строк. Соедините *transactions* со всеми остальными таблицами (*tr_mcc_codes*, *tr_types*, *gender_train*). Причём с *customers_gender_train* необходимо смёрджиться с помощью *left join*, а с оставшимися датафреймами – через *inner* (в результате соединения датафреймов должно получиться 999584 строки). Определите модуль разницы между средними тратами женщин и мужчин (трата – отрицательное значение *amount*). Выведите ответ в виде вещественного числа, округлённого до двух знаков после запятой, отделив дробную часть точкой в формате "123.45". Создайте новый столбец – *mcc_code + tr_type*, сложив значения из соответствующих столбцов.

Из всех типов транзакций оставьте только траты (отрицательные суммы по столбцу *amount*). Выделите из поля *tr_datetime* относительный день *tr_day* (первое число до точного времени). Измените тип поля *tr_day* на *int*. Выберите из *transactions* все МСС коды, которые встретились в выборке более чем 60000 раз. Сгруппируйте отфильтрованный датафрейм по дню и МСС-коду, получая средние значения суммы *amount*. Отрисуйте зависимость средних сумм (может пригодится метод *unstack()*) по каждому из МСС-кодов по дням. Проверьте верно ли, что 2 из полученных МСС-кодов связаны с финансовыми институтами.

3.1 Описание хода выполнения работы

```
import pandas as pd

tr_mcc_codes = pd.read_csv(filepath_or_buffer: "data/tr_mcc_codes.csv", sep=";")
transactions = pd.read_csv(filepath_or_buffer: "data/transactions.csv", sep=",", nrows=1_000_000)
tr_types = pd.read_csv(filepath_or_buffer: "data/tr_types.csv", sep=";")
gender_train = pd.read_csv(filepath_or_buffer: "data/gender_train.csv", sep=",")

df = transactions.merge(tr_mcc_codes, on="mcc_code", how="inner")
df = df.merge(tr_types, on="tr_type", how="inner")
df = df.merge(gender_train, on="customer_id", how="left")

expenses = df[df["amount"] < 0]
avg_by_gender = expenses.groupby("gender")["amount"].mean().abs()
difference = round(abs(avg_by_gender.iloc[0] - avg_by_gender.iloc[1]), 2)

df["mcc_code+tr_type"] = df["mcc_code"].astype(str) + df["tr_type"].astype(str)
# df["mcc_code+tr_type"] = df["mcc_code"].astype(int) + df["tr_type"].astype(int)

print(difference)
print(df)
```

Рисунок 7 – нахождения модуля разницы между средними тратами женщин и мужчин.

На рисунке 7 показано считывание четырёх таблиц и соединение их в одну различными методами. После соединения из таблицы отсеиваются только отрицательные траты, которые сортируются по гендеру. Затем происходит подсчёт итого значения разницы средних трат между мужчинами и женщинами, а также добавление новой колонки с объединёнными значениями колонок *mcc_code* и *tr_type*.

```

import pandas as pd
import matplotlib.pyplot as plt

transactions = pd.read_csv(filepath_or_buffer="data/transactions.csv", sep=";", nrows=1_000_000)

transactions_expenses = transactions[transactions["amount"] < 0].copy()
print(transactions_expenses.head(10))

transactions_expenses["tr_day"] = transactions_expenses["tr_datetime"].str.split().str[0].astype(int)
print(transactions_expenses.head(10))

mcc_counts = transactions_expenses["mcc_code"].value_counts()
print(mcc_counts)
popular_mccs = mcc_counts[mcc_counts > 60_000].index
print(popular_mccs)

filtered = transactions_expenses[transactions_expenses["mcc_code"].isin(popular_mccs)]
print(filtered)

grouped = filtered.groupby(["tr_day", "mcc_code"])["amount"].mean().unstack()
print(grouped.head(10))

grouped.plot(figsize=(12, 6), title="Средние траты по MCC-кодам по дням")
plt.xlabel("День")
plt.ylabel("Средняя сумма трат")
plt.grid(True)
plt.tight_layout()
plt.show()

financial = [6011, 6012]
found_financial = [i for i in grouped.columns if i in financial]

print("Связанные с фин. институтами:", found_financial)

```

Рисунок 8 – нахождение зависимости средних сумм по каждому из MCC-кодов по дням.

На рисунке 8 показан процесс анализа данных таблицы транзакций. Мы оставляем только отрицательные транзакции, выводим для отладки первые 10 строк. Затем выделяем день в отдельный столбец и присоединяем его справа, также делаем отладочный вывод. Находим самые популярные коды транзакций и выводим их. Затем создаём таблицу, где каждому коду транзакции и каждому дню соответствует количество использований этого кода в этот день. Формируем график на основе нашей таблицы и показываем его. В конце проверяем, сколько среди самых используемых кодов принадлежит к финансовым институтам.

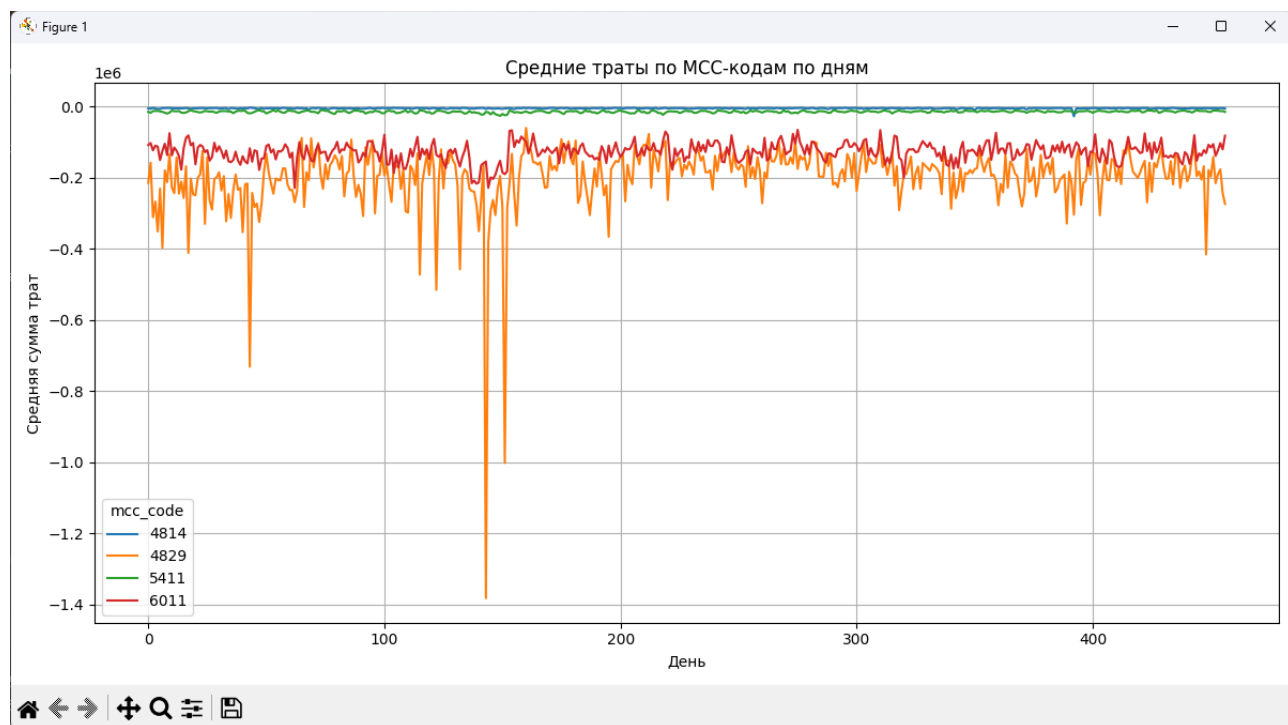


Рисунок 9 – вывод графика.

3.2 Вывод по разработке приложения

Разработанное приложение представляет собой эффективный инструмент первичного анализа данных. Благодаря четкой архитектуре его можно использовать в учебных целях

Освоены методики использования библиотек *pandas* и *matplotlib* для решения практических задач.

Научился разрабатывать модели компонентов информационных систем, включая модели баз данных, и модели интерфейсов «человек – электронно-вычислительная машина».

Заключение

В результате прохождения практики поставленная цель была достигнута, все задачи выполнены в полном объёме, приобретены необходимые профессиональные компетенции. Особое внимание было уделено разработке объектно-ориентированного приложения, что позволило на практике освоить ключевые принципы ООП.

Также в рамках практики был выполнен анализ транзакционных данных с использованием библиотек Python.

Я научился работать в коллективе (команде) при разработке программных средств (ОК-6). Знания, умения и навыки, полученные за период практики, стали личным стимулом для дальнейшего освоения специальности. Практика позволила применить теоретические знания, получить первый профессиональный опыт и сформировать представление о специфике будущей деятельности.

Список использованных источников

- 1) Канер С., Фолк Н., Нгуен Х. Тестирование программного обеспечения. Фундаментальные концепции: пер. с англ. — Москва: Вильямс, 2003. — 560 с.
- 2) Майерс Г. Искусство тестирования программ. — Москва: Издательство «Вильямс», 2004. — 256 с.
- 3) International Organization for Standardization. ISO/IEC/IEEE 29119: Software and Systems Engineering— Software Testing. — Женева: ISO, 2013.
- 4) Python Software Foundation. Documentation for unittest, pytest, coverage. — [Электронный ресурс]. — Режим доступа: <https://docs.python.org>, <https://docs.pytest.org>, <https://coverage.readthedocs.io> (дата обращения: 01.07.2025).
- 5) Лапкин А.В. Автоматизация тестирования программного обеспечения. — СПб.: Питер, 2020. — 288 с.
- 6) Куликов Д.Ю. Тестирование программных систем. Практический подход. — М.: ДМК Пресс, 2022. — 352 с.
- 7) Freeman S., Pryce N. Growing Object-Oriented Software, Guided by Tests. — Addison-Wesley, 2009. — 384 p.
- 8) Cohn M. Succeeding with Agile: Software Development Using Scrum. — Addison-Wesley, 2009. — 504 p.

Приложение А

1. Код объектно-ориентированного приложения

```
class Car:
    def __init__(self, brand, model, year):
        self.brand = brand
        self.model = model
        self.year = year

    def display_info(self):
        info = f'Brand: {self.brand}, Model: {self.model}, Year: {self.year}'
        return info

    def matches_conditions(self, **conditions):
        matches = True
        if 'brand' in conditions and conditions['brand'].lower() != self.brand.lower():
            matches = False
        if 'model' in conditions and conditions['model'].lower() != self.model.lower():
            matches = False
        if 'year' in conditions and conditions['year'] != self.year:
            matches = False
        return matches

class PassengerCar(Car):
    def __init__(self, brand, model, year, body_type):
        super().__init__(brand, model, year)
        self.body_type = body_type

    def display_info(self):
        base_info = super().display_info()
        return f'{base_info}, Body Type: {self.body_type}'

    def matches_conditions(self, **conditions):
        matches = True
```



```

    if not super().matches_conditions(**conditions):
        matches = False
    if 'body_type' in conditions and conditions['body_type'].lower() !=
self.body_type.lower():
        matches = False
    return matches

```

```

class Truck(Car):
    def __init__(self, brand, model, year, load_capacity):
        super().__init__(brand, model, year)
        self.load_capacity = load_capacity # in tons

    def display_info(self):
        base_info = super().display_info()
        return f'{base_info}, Load Capacity: {self.load_capacity} tons'

    def matches_conditions(self, **conditions):
        matches = True
        if not super().matches_conditions(**conditions):
            matches = False
        if 'min_load_capacity' in conditions and self.load_capacity <
conditions['min_load_capacity']:
            matches = False
        return matches

```

```

class Bus(Car):
    def __init__(self, brand, model, year, seat_count):
        super().__init__(brand, model, year)
        self.seat_count = seat_count

    def display_info(self):
        base_info = super().display_info()
        return f'{base_info}, Seat Count: {self.seat_count}'

```

```

def matches_conditions(self, **conditions):
    matches = True
    if not super().matches_conditions(**conditions):
        matches = False
    if 'min_seat_count' in conditions and self.seat_count <
conditions['min_seat_count']:
        matches = False
    return matches

```

```

vehicles = [
    PassengerCar("Audi"φ, "SQ7", 2018, "SUV"),
    PassengerCar("Volkswagen", "Passat", 2023, "sedan"),
    Truck("Volvo", "FH16", 2015, 20),
    Truck("Scania", "R450", 2019, 18),
    Bus("Mercedes", "Sprinter", 2017, 20),
    Bus("Volvo", "B9R", 2014, 50)
]

```

```

print("Все машины:")
for vehicle in vehicles:
    print(vehicle.display_info())

```

```

search_brand = "Volvo"
print(f"\nМашины бренда - '{search_brand}':")
for vehicle in vehicles:
    if vehicle.brand.lower() == search_brand.lower():
        print(vehicle.display_info())

```

```

search_year = 2019
print(f"\nМашины года - {search_year}:")
for vehicle in vehicles:
    if vehicle.year == search_year:
        print(vehicle.display_info())

```

2. Код первой задачи для анализа данных

```
import pandas as pd

tr_mcc_codes = pd.read_csv("data/tr_mcc_codes.csv", sep=";")
transactions = pd.read_csv("data/transactions.csv", sep=";", nrows=1_000_000)
tr_types = pd.read_csv("data/tr_types.csv", sep=";")
gender_train = pd.read_csv("data/gender_train.csv", sep=";",)

df = transactions.merge(tr_mcc_codes, on="mcc_code", how="inner")
df = df.merge(tr_types, on="tr_type", how="inner")
df = df.merge(gender_train, on="customer_id", how="left")

expenses = df[df["amount"] < 0]
avg_by_gender = expenses.groupby("gender")["amount"].mean().abs()
difference = round(abs(avg_by_gender.iloc[0] - avg_by_gender.iloc[1]), 2)

df["mcc_code+tr_type"] = df["mcc_code"].astype(str) + df["tr_type"].astype(str)
# df["mcc_code+tr_type"] = df["mcc_code"].astype(int) + df["tr_type"].astype(int)

print(difference)
print(df)
```

3. Код второй задачи для анализа данных

```
import pandas as pd
import matplotlib.pyplot as plt

transactions = pd.read_csv("data/transactions.csv", sep=";", nrows=1_000_000)

transactions_expenses = transactions[transactions["amount"] < 0].copy()
print(transactions_expenses.head(10))

transactions_expenses["tr_day"] = transactions_expenses["tr_datetime"].str.split().str[0].astype(int)
print(transactions_expenses.head(10))
```

```

mcc_counts = transactions_expenses["mcc_code"].value_counts()
print(mcc_counts)
popular_mccs = mcc_counts[mcc_counts > 60_000].index
print(popular_mccs)

filtered =
transactions_expenses[transactions_expenses["mcc_code"].isin(popular_mccs)]
print(filtered)

grouped = filtered.groupby(["tr_day", "mcc_code"])["amount"].mean().unstack()
print(grouped.head(10))

grouped.plot(figsize=(12, 6), title="Средние траты по МСС-кодам по дням")
plt.xlabel("День")
plt.ylabel("Средняя сумма трат")
plt.grid(True)
plt.tight_layout()
plt.show()

financial = [6011, 6012]
found_financial = [i for i in grouped.columns if i in financial]

print("Связанные с фин. институтами:", found_financial)

```